Video Streaming Service σε Multi-node Kubernetes

Αρχιτεκτονικές Δικτύων Επόμενης Γενιάς – Αναλυτική επεξήγηση όλων των μερών του project και των τεχνολογιών που χρησιμοποιούνται

Βασίζεται στον κώδικα του repository `Video-Streaming-Service` (branch `main`). Τα μέρη 1–3 είναι υλοποιημένα· για τα μέρη 4–6 περιγράφεται ο προτεινόμενος τρόπος υλοποίησης.

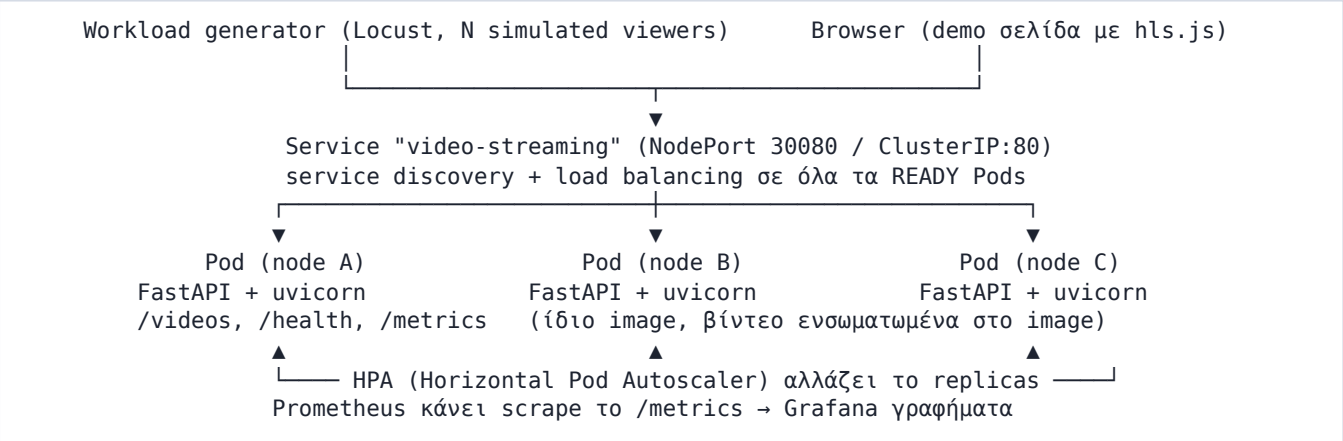
Περιεχόμενα

1. Συνολική εικόνα και αρχιτεκτονική
2. Μέρος 1 – Ανάπτυξη Video Streaming Service
3. Μέρος 2 – Containerization (Docker)
4. Μέρος 3 – Multi-node Kubernetes Deployment
5. Μέρος 4 – Workload Generator και Simulated Concurrent Users
6. Μέρος 5 – Horizontal Pod Autoscaling
7. Μέρος 6 – Monitoring και Performance Evaluation
8. Συνοπτικός πίνακας τεχνολογιών
9. Κατάσταση έργου, επόμενα βήματα και παρατηρήσεις

1. Συνολική εικόνα και αρχιτεκτονική

Ο στόχος της εργασίας δεν είναι το ίδιο το βίντεο, αλλά η **συμπεριφορά μιας υπηρεσίας streaming όταν μεταβάλλεται ο αριθμός των ταυτόχρονων θεατών**: πώς κατανέμεται το φορτίο σε πολλά αντίγραφα (replicas), πώς το Kubernetes προσθέτει ή αφαιρεί αντίγραφα αυτόματα, και πώς αυτό φαίνεται σε μετρήσεις (latency, throughput, CPU, αριθμός Pods).

Η βασική σχεδιαστική απόφαση είναι ότι η υπηρεσία είναι **stateless**: δεν κρατά καμία πληροφορία για τους χρήστες· απλώς σερβίρει αρχεία μέσω HTTP. Έτσι οποιοδήποτε Pod μπορεί να απαντήσει σε οποιοδήποτε αίτημα, και η οριζόντια κλιμάκωση (περισσότερα ίδια Pods) λειτουργεί χωρίς συγχρονισμό μεταξύ τους.



2. Μέρος 1 – Ανάπτυξη Video Streaming Service Υλοποιημένο

2.1 Επιλογή streaming protocol: HLS (HTTP Live Streaming)

Το HLS είναι πρωτόκολλο της Apple, σήμερα το πιο διαδεδομένο μαζί με το MPEG-DASH. Το βίντεο κόβεται εκ των προτέρων σε μικρά κομμάτια (**segments**) διάρκειας λίγων δευτερολέπτων και δημιουργείται ένα αρχείο-λίστα (**playlist**, `index.m3u8`) που περιγράφει τη σειρά τους. Ο client κατεβάζει πρώτα το playlist και μετά τα segments ένα-ένα με απλά HTTP GET.

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-TARGETDURATION:4          ← μέγιστη διάρκεια segment (4 s)
#EXT-X-PLAYLIST-TYPE:VOD         ← Video on Demand: πλήρης, αμετάβλητη λίστα
#EXTINF:4.000000,
seg_000.ts                       ← 1ο segment (MPEG-TS)
#EXTINF:4.000000,
seg_001.ts
...
#EXT-X-ENDLIST
```

Γιατί HLS και όχι π.χ. RTMP/RTSP/WebRTC:

- **Βασίζεται σε απλό HTTP**: κάθε αίτημα είναι ανεξάρτητο, άρα ένας load balancer μπορεί να στείλει κάθε segment σε διαφορετικό Pod. Αυτό είναι ακριβώς ό,τι χρειάζεται για να μελετηθεί η κλιμάκωση.
- **Stateless server**: ο server δεν κρατά συνεδρία (session) για κάθε θεατή, σε αντίθεση με τα RTSP/WebRTC.
- **Ρεαλιστικό**: έτσι λειτουργούν Netflix, YouTube, Twitch κ.ά. (HLS/DASH πάνω σε HTTP).
- **Μετρήσιμο**: κάθε segment είναι ένα αίτημα με γνωστό μέγεθος, οπότε μετράμε εύκολα request rate, throughput και latency.

2.2 Προετοιμασία του βίντεο: ffmpeg και `scripts/prepare_media.py`

Το **ffmpeg** είναι το κλασικό εργαλείο γραμμής εντολών για μετατροπή βίντεο. Το script `prepare_media.py` το καλεί για να μετατρέψει ένα αρχείο (π.χ. MP4) σε HLS, μία φορά, πριν από το build. Έτσι ο server δεν κάνει κανένα transcoding την ώρα της αναπαραγωγής – μόνο μεταφορά αρχείων.

Ρύθμιση ffmpeg	Τι κάνει / γιατί
`-c:v libx264`, `-pix_fmt yuv420p`	Κωδικοποίηση H.264 8-bit 4:2:0, την οποία αποκωδικοποιούν όλοι οι browsers.
`-c:a aac -b:a 128k`	Ήχος AAC, το στάνταρ για HLS.
`-force_key_frames expr:gte(t,n_forced*4)`	Keyframe ακριβώς κάθε 4 s, ώστε κάθε segment να ξεκινά με keyframe και να έχει σταθερή διάρκεια.
`-hls_time 4`, `-hls_playlist_type vod`	Segments 4 δευτερολέπτων, πλήρες playlist με `#EXT-X-ENDLIST`.
`--test` (επιλογή του script)	Παράγει συνθετικό βίντεο (testsrc + ημιτονοειδής ήχος) χωρίς να χρειάζεται αρχείο.

Το αποτέλεσμα αποθηκεύεται στο `media/hls/<video_id>/` (`index.m3u8` + `seg_000.ts`, `seg_001.ts`, ...).

2.3 Web framework και server: FastAPI + uvicorn

- **FastAPI** (Python): σύγχρονο web framework για APIs. Ορίζουμε endpoints με decorators (`@router.get(...)`), κάνει αυτόματο έλεγχο παραμέτρων και παράγει αυτόματη τεκμηρίωση στο `/docs` (OpenAPI/Swagger).
- **uvicorn**: ASGI server, δηλαδή το πρόγραμμα που ακούει στη θύρα 8000 και περνά τα HTTP αιτήματα στο FastAPI. Είναι ασύγχρονος (asyncio), οπότε εξυπηρετεί πολλές ταυτόχρονες συνδέσεις σε μία διεργασία.
- **pydantic-settings**: διαβάζει τις ρυθμίσεις (`MEDIA_DIR`, `APP_NAME`, `INSTANCE_NAME`) από μεταβλητές περιβάλλοντος, ώστε το ίδιο image να τρέχει παντού χωρίς αλλαγές κώδικα.
- **prometheus-client**: εκθέτει μετρικές της εφαρμογής στο `/metrics` (βλ. Μέρος 6).

2.4 Δομή κώδικα

Αρχείο	Ρόλος
`app/main.py`	Δημιουργεί την εφαρμογή, συνδέει routers, middleware μετρικών, `/metrics`, στατικά αρχεία και τη σελίδα `/`.
`app/api/routes/videos.py`	Endpoints `GET /videos` και `GET /videos/{id}/{file}`.
`app/api/routes/health.py`	Health checks `/health/live`, `/health/ready`.
`app/services/video_service.py`	Εύρεση αρχείων βίντεο και προστασία από path traversal.
`app/core/config.py`	Ρυθμίσεις από μεταβλητές περιβάλλοντος / `.env`.
`app/core/metrics.py`	Ορισμός μετρικών Prometheus και middleware που τις ενημερώνει σε κάθε αίτημα.
`app/static/`	Demo σελίδα (HTML/CSS/JS) με player hls.js.

2.5 Endpoints

Endpoint	Περιγραφή
`GET /videos`	Λίστα διαθέσιμων βίντεο (JSON) με το URL του playlist κάθε βίντεο.
`GET /videos/{id}/index.m3u8`	Το playlist ενός βίντεο (`application/vnd.apple.mpegurl`).
`GET /videos/{id}/seg_NNN.ts`	Ένα segment (`video/mp2t`). Αυτό είναι το «βαρύ» αίτημα που παράγει το φορτίο.
`GET /health/live`	Liveness: η διεργασία ζει.
`GET /health/ready`	Readiness: υπάρχει τουλάχιστον ένα βίντεο. Αλλιώς 503.
`GET /metrics`	Μετρικές σε μορφή Prometheus.
`GET /`, `/docs`	Demo σελίδα και αυτόματη τεκμηρίωση API.

Κάθε απάντηση βίντεο φέρει την κεφαλίδα `X-Served-By` με το όνομα του Pod που την εξυπηρέτησε. Έτσι στη demo σελίδα και στον workload generator φαίνεται ότι τα segments μοιράζονται σε διαφορετικά Pods.

2.6 Ασφάλεια: προστασία από path traversal

Επειδή το URL περιέχει ονόματα αρχείων, ένας κακόβουλος client θα μπορούσε να ζητήσει π.χ. `/videos/../../etc/passwd`. Το `video_service.py` το αποτρέπει με τρία επίπεδα: (α) το id επιτρέπεται

να έχει μόνο γράμματα, αριθμούς, `_` και `-`, (β) το όνομα αρχείου πρέπει να τελειώνει σε `.m3u8` ή `.ts`, (γ) η τελική διαδρομή ελέγχεται ότι βρίσκεται μέσα στον φάκελο HLS (`is_relative_to`). Οτιδήποτε άλλο επιστρέφει 404.

2.7 Demo σελίδα και hls.js

Οι Chrome/Firefox/Edge δεν παίζουν HLS εγγενώς. Η βιβλιοθήκη JavaScript **hls.js** κατεβάζει playlist και segments, μετατρέπει (transmux) το MPEG-TS σε fragmented MP4 και τα δίνει στο στοιχείο `<video>` μέσω του Media Source Extensions API. Η σελίδα καταγράφει για κάθε segment το μέγεθος, τον χρόνο λήψης και το Pod που το σέρβιρε. Η σελίδα χρησιμεύει μόνο για επίδειξη· ο workload generator δεν τη χρειάζεται.

3. Μέρος 2 – Containerization (Docker) Υλοποιημένο

3.1 Τι είναι το Docker και γιατί χρειάζεται

Ένα **container image** είναι ένα «πακέτο» που περιέχει την εφαρμογή μαζί με όλα όσα χρειάζεται (Python, βιβλιοθήκες, αρχεία βίντεο). Ένα **container** είναι μια εκτέλεση αυτού του image, απομονωμένη από το υπόλοιπο σύστημα. Το πλεονέκτημα είναι ότι το ίδιο image τρέχει ίδια σε κάθε μηχάνημα: στον υπολογιστή μας, σε άλλου μέλους της ομάδας και στους κόμβους του Kubernetes, όπως ζητά η εκφώνηση.

3.2 Το Dockerfile βήμα-βήμα

```
FROM python:3.13-slim                       # μικρή επίσημη εικόνα Python (Debian)
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 MEDIA_DIR=/app/media
WORKDIR /app
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt   # πρώτα οι εξαρτήσεις → cache
RUN useradd --create-home --uid 1000 appuser       # χρήστης χωρίς δικαιώματα root
COPY backend/app ./app                              # κώδικας
COPY media/hls ./media/hls                          # έτοιμα HLS βίντεο
USER appuser
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- **Σειρά των layers**: το `requirements.txt` αντιγράφεται και εγκαθίσταται πριν από τον κώδικα. Έτσι, όταν αλλάζει μόνο ο κώδικας, το Docker ξαναχρησιμοποιεί το layer με τις βιβλιοθήκες και το build είναι πολύ γρηγορότερο.
- **Μη-root χρήστης**: αν κάποιος εκμεταλλευτεί κενό ασφαλείας, δεν έχει δικαιώματα διαχειριστή μέσα στο container.
- **Βίντεο μέσα στο image**: κάθε Pod έχει δικό του αντίγραφο των αρχείων, χωρίς κοινόχρηστο δίσκο (volume). Αυτό κρατά τα Pods πλήρως ανεξάρτητα και stateless, κάτι που απλοποιεί την κλιμάκωση. Το κόστος είναι ότι για νέο βίντεο χρειάζεται νέο image.
- **`--host 0.0.0.0`**: ο server ακούει σε όλες τις διεπαφές δικτύου, όχι μόνο στο localhost του container, ώστε να είναι προσβάσιμος απ' έξω.
- **`.dockerignore`**: εξαιρεί από το build context περιττά αρχεία (`.git`, `.venv`, `media/source`), ώστε το image να μένει μικρό.

3.3 Εκτέλεση και επαλήθευση

```
python scripts/prepare_media.py --test          # δημιουργία βίντεο (αν δεν υπάρχει)
docker build -t video-streaming:1.0 .
docker run --rm -p 8000:8000 video-streaming:1.0
curl http://localhost:8000/health/ready
curl -I http://localhost:8000/videos/test/seg_000.ts   # Content-Type: video/mp2t, X-Served-By
```

Το `-p 8000:8000` αντιστοιχίζει τη θύρα 8000 του υπολογιστή στη θύρα 8000 του container. Η αναπαραγωγή επιβεβαιώνεται ανοίγοντας το `http://localhost:8000/`.

4. Μέρος 3 – Multi-node Kubernetes Deployment Υλοποιημένο

4.1 Τι είναι το Kubernetes

Το **Kubernetes** είναι σύστημα ενορχήστρωσης containers: του δηλώνουμε την επιθυμητή κατάσταση (π.χ. «3 αντίγραφα αυτού του image») και αυτό φροντίζει συνεχώς να την πετυχαίνει. Τοποθετεί containers σε κόμβους, τα επανεκκινεί αν πέσουν, τα συνδέει στο δίκτυο και τα κλιμακώνει. Βασικές έννοιες:

- **Node**: ένα μηχάνημα (εδώ ένα container του Docker) που τρέχει Pods.
- **Pod**: η μικρότερη μονάδα εκτέλεσης· εδώ ένα Pod = ένα container της υπηρεσίας.
- **Deployment**: διατηρεί σταθερό αριθμό ίδιων Pods και κάνει rolling updates.
- **Service**: σταθερή διεύθυνση δικτύου μπροστά από ένα μεταβαλλόμενο σύνολο Pods.
- **kubectl**: το εργαλείο γραμμής εντολών για επικοινωνία με το cluster.

4.2 Τοπικό multi-node cluster με minikube

Το **minikube** δημιουργεί τοπικό cluster. Με τον Docker driver κάθε κόμβος είναι ένα container του Docker Desktop:

```
minikube start --nodes 3 --driver=docker --cpus 2 --memory 2200
kubectl get nodes          # minikube (control-plane), minikube-m02, minikube-m03
```

Επειδή το cluster δεν βλέπει τα images του υπολογιστή, το image φορτώνεται στους κόμβους με `minikube image load video-streaming:1.0`. Γι' αυτό το Deployment έχει `imagePullPolicy: IfNotPresent`: χρησιμοποιεί το τοπικό image και δεν προσπαθεί να το κατεβάσει από το Docker Hub. Σε κάθε αλλαγή χρησιμοποιείται **νέο tag** (1.1, 1.2, ...), αλλιώς οι κόμβοι κρατούν το παλιό image.

4.3 Deployment (`k8s/deployment.yaml`) – μηχανισμοί και αιτιολόγηση

Μηχανισμός	Ρύθμιση	Αιτιολόγηση
Replicas	`replicas: 3`	Ένα Pod ανά κόμβο· με το HPA αυτός ο αριθμός θα αλλάζει αυτόματα.
Resource requests	CPU 100m, μνήμη 128Mi	Ο scheduler δεσμεύει αυτούς τους πόρους για κάθε Pod και τοποθετεί Pods μόνο σε κόμβους που τους έχουν. Επίσης, τα ποσοστά του HPA υπολογίζονται **ως προς το request** (π.χ. 50% = 50m CPU).
Resource limits	CPU 500m, μνήμη 256Mi	Ένα Pod δεν μπορεί να «φάει» όλο τον κόμβο. Η CPU πάνω από το όριο περιορίζεται (throttling)· η μνήμη πάνω από το όριο οδηγεί σε επανεκκίνηση (OOMKilled).

Readiness probe	`GET /health/ready` κάθε 5 s	Το Pod δέχεται κίνηση από το Service μόνο όταν μπορεί πραγματικά να σερβίρει βίντεο. Κατά το scale-out, νέα Pods δεν λαμβάνουν αιτήματα πριν είναι έτοιμα.
Liveness probe	`GET /health/live` κάθε 10 s	Αν η διεργασία κολλήσει (3 αποτυχίες), το Kubernetes επανεκκινεί το container.
Workload placement	`topologySpreadConstraints`, `maxSkew: 1` ανά `kubernetes.io/hostname`	Τα Pods μοιράζονται όσο πιο ομοιόμορφα γίνεται στους κόμβους (διαφορά το πολύ 1). Αυτό δίνει ανοχή σε αποτυχία κόμβου και καλύτερη αξιοποίηση. Με `ScheduleAnyway` το Pod τοποθετείται ακόμη κι αν η τέλεια κατανομή δεν είναι δυνατή, αντί να μείνει Pending.

Γιατί δύο διαφορετικά probes; Το readiness απαντά στο «να του στείλω κίνηση;» και το liveness στο «να το επανεκκινήσω;». Αν ένα Pod δεν έχει βίντεο, δεν ωφελεί η επανεκκίνηση (θα συνεχίσει να μην έχει)· αρκεί να μη δέχεται κίνηση.

4.4 Service (`k8s/service.yaml`) – service discovery και load balancing

- **Selector** `app: video-streaming`: το Service βρίσκει δυναμικά όλα τα Pods με αυτή την ετικέτα. Όταν το HPA προσθέτει ή αφαιρεί Pods, η λίστα (Endpoints) ενημερώνεται αυτόματα.
- **Service discovery μέσω DNS**: μέσα στο cluster το Service είναι προσβάσιμο ως `http://video-streaming` (CoreDNS). Έτσι, ο workload generator, αν τρέξει μέσα στο cluster, δεν χρειάζεται IP διευθύνσεις.
- **Load balancing**: το **kube-proxy** σε κάθε κόμβο μοιράζει τις νέες συνδέσεις στα READY Pods.
- `type: NodePort`, `nodePort: 30080`: η υπηρεσία είναι προσβάσιμη και εκτός cluster στη θύρα 30080 κάθε κόμβου. Στα Windows με Docker driver χρησιμοποιείται `minikube service video-streaming --url`, που ανοίγει tunnel.

Σημαντική λεπτομέρεια για τα πειράματα: το kube-proxy ισοκατανέμει **συνδέσεις TCP**, όχι μεμονωμένα αιτήματα. Ένας client που κρατά ανοιχτή σύνδεση (HTTP keep-alive) στέλνει όλα του τα segments στο ίδιο Pod. Με πολλούς simulated users, ο καθένας με δική του σύνδεση, το φορτίο μοιράζεται σωστά· αυτό πρέπει να ληφθεί υπόψη στον workload generator.

5. Μέρος 4 – Workload Generator και Simulated Concurrent Users

Προς υλοποίηση

5.1 Πώς συμπεριφέρεται ένας πραγματικός HLS client

Για να είναι το φορτίο αντιπροσωπευτικό, ο simulated user πρέπει να κάνει ακριβώς τα δικτυακά αιτήματα που κάνει ένας player όπως ο hls.js:

1. Ζητά `GET /videos/<id>/index.m3u8` και διαβάζει τη λίστα των segments.
2. **Φάση εκκίνησης (buffering)**: κατεβάζει γρήγορα τα πρώτα 2–3 segments χωρίς αναμονή, για να γεμίσει το buffer.
3. **Φάση αναπαραγωγής (steady state)**: κατεβάζει περίπου ένα segment ανά διάρκεια segment (4 s), δηλαδή με τον ρυθμό που «καταναλώνεται» το βίντεο. Αν η λήψη αργήσει, ο επόμενος κύκλος ξεκινά αμέσως (ο player προσπαθεί να προλάβει).

4. Στο τέλος του βίντεο ξεκινά από την αρχή (ή επιλέγει άλλο βίντεο), ώστε ο θεατής να παραμένει ενεργός όσο διαρκεί η φάση.

Δεν χρειάζεται browser ή αποκωδικοποίηση εικόνας: για τον server το φορτίο είναι τα HTTP αιτήματα και τα bytes που στέλνει. Η αποκωδικοποίηση γίνεται στη συσκευή του θεατή και δεν επηρεάζει την υπηρεσία.

5.2 Προτεινόμενο εργαλείο: Locust

Το **Locust** είναι εργαλείο load testing σε Python. Κάθε simulated user είναι ένα ελαφρύ coroutine (gevent), οπότε ένας υπολογιστής προσομοιώνει εκατοντάδες θεατές. Παρέχει web UI με γραφήματα σε πραγματικό χρόνο, εξαγωγή αποτελεσμάτων σε CSV και, κυρίως, το `LoadTestShape` για προγραμματισμένες φάσεις φόρτου. Εναλλακτικές: k6 (JavaScript), JMeter, ή δικό μας script με `asyncio` + `httpx`.

```
from locust import HttpUser, LoadTestShape, task, constant
import time

SEGMENT_SECONDS = 4

class Viewer(HttpUser):
    wait_time = constant(0)

    @task
    def watch(self):
        playlist = self.client.get("/videos/test/index.m3u8", name="playlist").text
        segments = [l for l in playlist.splitlines() if l.endswith(".ts")]
        for i, seg in enumerate(segments):
            start = time.time()
            self.client.get(f"/videos/test/{seg}", name="segment")
            if i >= 2:                                  # μετά το αρχικό buffering
                time.sleep(max(0, SEGMENT_SECONDS - (time.time() - start)))

class Phases(LoadTestShape):
    # (τέλος φάσης σε s, θεατές, ρυθμός προσθήκης/αφαίρεσης ανά s)
    stages = [(120, 20, 2), (300, 100, 5), (480, 200, 5),   # αύξηση ζήτησης
              (660, 100, 5), (840, 20, 5), (960, 0, 5)]     # μείωση ζήτησης
    def tick(self):
        t = self.get_run_time()
        for end, users, rate in self.stages:
            if t < end:
                return users, rate
        return None
```

Εκτέλεση: `locust -f loadtest/locustfile.py --host http://video-streaming --headless --csv results`. Τα βήματα του `stages` υλοποιούν την απαίτηση για **φάσεις αύξησης και μείωσης** της ζήτησης.

5.3 Πού τρέχει ο generator

Συνιστάται ο generator να τρέχει **μέσα στο cluster** ως Pod (ή Job) και να στοχεύει το `http://video-streaming`. Στα Windows, το tunnel του `minikube service` είναι μία διεργασία που περνά όλη την κίνηση και γίνεται εύκολα το bottleneck, οπότε θα μετρούσαμε το tunnel και όχι την υπηρεσία. Αν τρέξει εκτός cluster, πρέπει να καταγραφεί ως περιορισμός στην αναφορά.

5.4 Γιατί το workload είναι αντιπροσωπευτικό

- Κάνει **τα ίδια αιτήματα με την ίδια σειρά** με πραγματικό player (playlist → segments).

- Τηρεί τον **χρονισμό** της αναπαραγωγής (1 segment / 4 s μετά το buffering), άρα κάθε θεατής καταναλώνει bandwidth ίσο με το bitrate του βίντεο, όπως στην πραγματικότητα.
- Κατεβάζει **πραγματικά bytes** (όλο το segment), οπότε επιβαρύνει δίκτυο, I/O και CPU του server.
- Κάθε θεατής έχει δική του σύνδεση, άρα η κατανομή φορτίου από το Service είναι ρεαλιστική.
- **Περιορισμοί**: δεν προσομοιώνεται adaptive bitrate (υπάρχει μία ποιότητα), ούτε συμπεριφορές όπως pause/seek. Τα 3 «nodes» μοιράζονται το ίδιο φυσικό μηχάνημα.

6. Μέρος 5 – Horizontal Pod Autoscaling Προς υλοποίηση

6.1 Πώς λειτουργεί το HPA

Ο **Horizontal Pod Autoscaler** είναι controller του Kubernetes που κάθε περίπου 15 s διαβάζει μια μετρική των Pods και αλλάζει τα `replicas` του Deployment με τον τύπο:

```
επιθυμητά replicas = ceil( τρέχοντα replicas × τρέχουσα τιμή μετρικής / τιμή-στόχος )
```

Για μετρικές CPU/μνήμης χρειάζεται ο **metrics-server**, που συλλέγει χρήση πόρων από το kubelet κάθε κόμβου. Στο minikube ενεργοποιείται με `minikube addons enable metrics-server`.

6.2 Προτεινόμενο manifest (`k8s/hpa.yaml`)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: video-streaming
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: video-streaming}
  minReplicas: 2            # διαθεσιμότητα: ποτέ λιγότερα από 2
  maxReplicas: 10           # όριο ώστε να χωρούν στους πόρους του cluster
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 60}   # 60% του request (60m)
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0        # γρήγορη αντίδραση στην αύξηση
      policies: [{type: Percent, value: 100, periodSeconds: 30}]
    scaleDown:
      stabilizationWindowSeconds: 120      # αργή μείωση: αποφυγή «ταλάντωσης»
      policies: [{type: Pods, value: 1, periodSeconds: 30}]
```

6.3 Επιλογή παραμέτρων

- **Στόχος 60%**: αφήνει περιθώριο για απότομες αυξήσεις κατά το διάστημα που χρειάζονται τα νέα Pods να γίνουν READY. Πολύ υψηλός στόχος (90%) σημαίνει καθυστερημένη αντίδραση· πολύ χαμηλός (30%) σπατάλη πόρων.
- **min/max replicas**: το min εξασφαλίζει διαθεσιμότητα· το max πρέπει να χωρά στη μνήμη και CPU του cluster (requests × max ≤ διαθέσιμοι πόροι), αλλιώς τα Pods μένουν Pending.
- **Ασυμμετρία scale-up / scale-down**: γρήγορη αύξηση για να μην πέσει η ποιότητα υπηρεσίας, αργή μείωση ώστε στιγμιαίες πτώσεις φόρτου να μην αφαιρούν Pods που θα ξαναχρειαστούν σε λίγο (flapping).
- Οι παράμετροι επιλέγονται **πειραματικά**: πρώτα μετράμε πόσους θεατές αντέχει ένα Pod (π.χ. ώσπου το p95 latency των segments να ξεπεράσει κάποιο όριο) και πόση CPU αντιστοιχεί σε

αυτούς.

6.4 Είναι η CPU κατάλληλο scaling signal για video streaming;

Αυτό είναι το κεντρικό ερώτημα της ενότητας. Η σερβιρισμένη ροή βίντεο είναι κυρίως φορτίο **δικτύου και I/O**: ο server διαβάζει αρχεία από τον δίσκο και τα στέλνει στο δίκτυο, χωρίς transcoding.

Υπέρ της CPU	Κατά της CPU
• Διαθέσιμη χωρίς επιπλέον υποδομή (metrics-server). • Στη δική μας υλοποίηση (Python/uvicorn) η αποστολή κάθε chunk περνά από τον interpreter και το middleware μετρικών, οπότε η CPU **αυξάνεται περίπου ανάλογα** με τα bytes/s. • Το CPU limit (500m) είναι πραγματικό όριο: όταν φτάνει, το throttling αυξάνει το latency.	• Ο πραγματικός πόρος που εξαντλείται συχνά είναι το **bandwidth** του κόμβου, όχι η CPU. Ένας αποδοτικός server (π.χ. nginx με sendfile) θα είχε σχεδόν μηδενική CPU ενώ το δίκτυο είναι κορεσμένο. • Η CPU δεν δείχνει άμεσα τον αριθμό των **ενεργών θεατών** ή την ποιότητα εμπειρίας (π.χ. αν τα segments φτάνουν σε λιγότερο από 4 s). • Εξαρτάται από το μέγεθος/bitrate του βίντεο: ίδιοι θεατές με βίντεο 1080p δίνουν πολύ περισσότερη CPU από ό,τι με 480p.

Εναλλακτικές μετρικές που εκφράζουν πιο άμεσα το φορτίο: `http_requests_in_progress` (ταυτόχρονες μεταφορές), throughput (bytes/s ανά Pod) ή αιτήματα segments/s ανά Pod. Αυτές μπορούν να τροφοδοτήσουν το HPA ως `type: Pods` custom metrics μέσω του **Prometheus Adapter** ή του **KEDA**. Μια καλή πειραματική πρόταση είναι να εκτελεστεί το ίδιο σενάριο με CPU-based HPA και να συγκριθεί η συσχέτιση της CPU με τον αριθμό θεατών και το throughput στα γραφήματα. Αυτό απαντά τεκμηριωμένα στο ερώτημα της εκφώνησης.

7. Μέρος 6 – Monitoring και Performance Evaluation Εν μέρει υλοποιημένο

7.1 Μετρικές της εφαρμογής (ήδη υλοποιημένες στο `/metrics`)

Το middleware στο `app/core/metrics.py` τρέχει σε κάθε αίτημα και ενημερώνει τέσσερις μετρικές. Ο χρόνος και τα bytes μετρώνται όταν σταλεί και το τελευταίο byte της απάντησης, όχι μόνο οι κεφαλίδες. Αυτό είναι σημαντικό για αρχεία βίντεο, όπου η μεταφορά διαρκεί.

Μετρική	Τύπος	Τι μετρά
`http_requests_total`	Counter	Αιτήματα ανά method, route και status code → request rate και error rate.
`http_request_duration_seconds`	Histogram	Χρόνος εξυπηρέτησης → latency (μέσος, p95, p99).
`http_requests_in_progress`	Gauge	Ταυτόχρονες μεταφορές → ενεργοί θεατές ανά Pod.
`http_response_bytes_total`	Counter	Bytes που στάλθηκαν → throughput.

Τύποι μετρικών Prometheus: ο **Counter** μόνο αυξάνεται (από αυτόν υπολογίζουμε ρυθμούς με `rate()`), ο **Gauge** ανεβοκατεβαίνει, και το **Histogram** μετρά πόσες τιμές πέφτουν σε κάθε «κουβά» (bucket), ώστε να υπολογίζονται εκατοστημόρια (percentiles).

7.2 Προτεινόμενη στοίβα monitoring

- **Prometheus**: βάση δεδομένων χρονοσειρών που κάθε λίγα δευτερόλεπτα διαβάζει (scrape) το `/metrics` κάθε Pod. Τα Pods βρίσκονται αυτόματα μέσω του Kubernetes API (service discovery).

- **Grafana**: δημιουργία dashboards και γραφημάτων από τα δεδομένα του Prometheus.
- **kube-state-metrics**: μετρικές για αντικείμενα του Kubernetes (αριθμός replicas, επιθυμητά replicas του HPA).
- **cAdvisor / kubelet**: χρήση CPU, μνήμης και δικτύου ανά container.

Όλα εγκαθίστανται μαζί με το Helm chart **kube-prometheus-stack**:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
```

Στη συνέχεια ένα **ServiceMonitor** (ή PodMonitor) δηλώνει στον Prometheus να κάνει scrape τη θύρα `http` και το path `/metrics` των Pods με `app: video-streaming`. Πριν από αυτό πρέπει να ενεργοποιηθεί ο metrics-server (Μέρος 5).

7.3 Ενδεικτικά queries (PromQL) για τα γραφήματα

Γράφημα	PromQL
Request rate (segments/s)	`sum(rate(http_requests_total{path="/videos/{video_id}/{filename}"}[1m]))`
Throughput (Mbit/s)	`sum(rate(http_response_bytes_total[1m])) * 8 / 1e6`
p95 latency segments	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[1m])))`
Error rate	`sum(rate(http_requests_total{status=~"5.."}[1m]))`
Ενεργές μεταφορές	`sum(http_requests_in_progress)`
CPU ανά Pod	`sum by (pod) (rate(container_cpu_usage_seconds_total{pod=~"video-streaming.*"}[1m]))`
Replicas (τρέχοντα/ επιθυμητά)	`kube_deployment_status_replicas{deployment="video-streaming"}`, `kube_horizontalpodautoscaler_status_desired_replicas`

7.4 Παρουσίαση και συσχέτιση αποτελεσμάτων

Το κύριο γράφημα της αναφοράς είναι μια κοινή χρονική γραμμή που δείχνει μαζί: **αριθμό simulated θεατών** (από Locust), **CPU**, **replicas**, **throughput** και **p95 latency**. Πάνω σε αυτή εξηγούμε:

- Πόση καθυστέρηση υπάρχει από την αύξηση των θεατών μέχρι το scale-out (χρόνος αντίδρασης HPA + εκκίνηση/readiness νέων Pods).
- Αν στο διάστημα αυτό αυξάνεται το latency ή εμφανίζονται σφάλματα.
- Αν η CPU ακολουθεί πιστά τον αριθμό θεατών/throughput (απάντηση στο ερώτημα του Μέρους 5).
- Πόσο αργά γίνεται το scale-in μετά τη μείωση φόρτου και γιατί (stabilization window).
- Αν η κίνηση μοιράζεται ισόποσα στα Pods (CPU/throughput ανά Pod, κεφαλίδα `X-Served-By`).

Για την αναφορά, τα δεδομένα μπορούν να εξαχθούν από τον Prometheus (HTTP API) και από τα CSV του Locust και να σχεδιαστούν με matplotlib, ή να χρησιμοποιηθούν screenshots από το Grafana.

8. Συνοπτικός πίνακας τεχνολογιών

Τεχνολογία	Κατηγορία	Χρήση στο project

HLS (m3u8 + MPEG-TS)	Streaming protocol	Το βίντεο σε segments των 4 s, μεταδιδόμενα με απλό HTTP· επιτρέπει stateless κλιμάκωση.
ffmpeg	Επεξεργασία βίντεο	Μετατροπή του βίντεο σε HLS (H.264/AAC) πριν από το build.
Python 3.13	Γλώσσα	Υλοποίηση της υπηρεσίας και των scripts.
FastAPI	Web framework	Endpoints, έλεγχος παραμέτρων, αυτόματη τεκμηρίωση.
uvicorn	ASGI web server	Ακούει στη θύρα 8000 και εξυπηρετεί ασύγχρονα πολλές συνδέσεις.
pydantic-settings	Ρυθμίσεις	Ρυθμίσεις από μεταβλητές περιβάλλοντος → ίδιο image παντού.
hls.js	JavaScript player	Αναπαραγωγή HLS στη demo σελίδα (Chrome/Firefox/Edge).
Docker / Docker Desktop	Containerization	Κατασκευή και τοπική εκτέλεση του image· οι κόμβοι του minikube τρέχουν ως containers.
Kubernetes	Orchestration	Deployment, Service, probes, resources, placement, HPA.
minikube	Τοπικό cluster	Δημιουργία multi-node cluster 3 κόμβων στον υπολογιστή.
kubectl	CLI	Εφαρμογή manifests, παρακολούθηση Pods/κόμβων, logs.
kube-proxy, CoreDNS	Δικτύωση K8s	Load balancing του Service και service discovery με όνομα.
metrics-server	Μετρικές πόρων	Τροφοδοτεί το HPA με χρήση CPU/μνήμης.
Locust	Load testing	Simulated concurrent viewers με φάσεις φόρτου.
prometheus-client	Instrumentation	Μετρικές της εφαρμογής στο `/metrics`.
Prometheus	Monitoring	Συλλογή και αποθήκευση χρονοσειρών μετρικών.
Grafana	Οπτικοποίηση	Dashboards και γραφήματα για την αξιολόγηση.
Helm	Package manager K8s	Εγκατάσταση του kube-prometheus-stack.
Git / GitHub	Έλεγχος εκδόσεων	Συνεργασία της ομάδας, κοινός κώδικας και βίντεο.

9. Κατάσταση έργου, επόμενα βήματα και παρατηρήσεις

Βήμα	Κατάσταση	Επόμενη ενέργεια
1. Video streaming service	Ολοκληρωμένο	—
2. Docker	Ολοκληρωμένο	—
3. Kubernetes multi-node	Σε εξέλιξη	Επιβεβαίωση ότι τα Pods μοιράζονται σε 3 κόμβους (`kubectl get pods -o wide`).
4. Workload generator	Εκκρεμεί	`loadtest/locustfile.py` με φάσεις, εκτέλεση μέσα στο cluster.
5. HPA	Εκκρεμεί	`minikube addons enable metrics-server`, `k8s/hpa.yaml`, πειράματα.
6. Monitoring	Μετρικές εφαρμογής έτοιμες	kube-prometheus-stack, ServiceMonitor, dashboard, γραφήματα.

Παρατήρηση στον κώδικα: στο `app/core/metrics.py` ο counter ονομάζεται `"https_requests_total"` (με «s»), ενώ το README και τα queries χρησιμοποιούν `http_requests_total`. Πρέπει να διορθωθεί σε `http_requests_total` πριν από το Μέρος 6, αλλιώς τα queries δεν θα βρίσκουν δεδομένα. Επίσης, η περιγραφή του `http_request_duration_seconds` είναι αντιγραμμένη από τον counter («Total amount of HTTP requests») και καλό είναι να γίνει «Request duration in seconds».

Συνοπτικά: η υπηρεσία σερβίρει HLS segments μέσω HTTP από stateless Pods. Το Kubernetes τα κατανέμει σε κόμβους, τα ελέγχει με probes και ισοκατανέμει την κίνηση μέσω του Service. Ο Locust παράγει ρεαλιστικό φορτίο θεατών σε φάσεις, το HPA προσαρμόζει τον αριθμό των Pods, και ο συνδυασμός Prometheus + Grafana δείχνει πώς το φορτίο, οι αποφάσεις κλιμάκωσης και η απόδοση συσχετίζονται μεταξύ τους.